

NSU ClubHub: A Centralized Club Management & Engagement Portal for North South University

M.A. Rahman Junaid

Dept. of ECE

North South University

Dhaka, Bangladesh

ID: 2311947642

Shabab Alam Pean

Dept. of ECE

North South University

Dhaka, Bangladesh

ID: 2231829042

Mohammad Safayet Alif

Dept. of ECE

North South University

Dhaka, Bangladesh

ID: 2232562642

Syed Mohim Rahman Preo

Dept. of ECE

North South University

Dhaka, Bangladesh

ID: 2232812642

Dr. Mohammad Shifat-E-Rabbi

Faculty Advisor

Assistant Professor, Dept. of ECE

North South University, Dhaka, Bangladesh

Spring 2026

Abstract—University clubs play a vital role in student development; however, traditional club management at North South University (NSU) has relied on fragmented, informal channels, making participation and oversight difficult for students and administrators alike. This paper presents *NSU ClubHub*, a full-stack web portal that consolidates club discovery, event management, QR-based ticketing, recruitment handling, real-time notifications, and an AI-powered assistant into a single unified platform. The frontend is implemented with Next.js 15 and Tailwind CSS; the backend uses Express.js and TypeScript; and all data is persisted in a PostgreSQL relational database accessed through Prisma ORM. The system enforces JWT-based authentication and role-based access control (RBAC) across three user roles: Student, Club Administrator, and Super Administrator. Functional testing verified correct behaviour across all user flows, security testing confirmed proper access enforcement, and initial usability sessions with NSU students validated the platform’s accessibility and ease of use. NSU ClubHub provides a scalable and secure template for modernising co-curricular management at any university.

Index Terms—club management system, university portal, full-stack web application, Next.js, Express.js, PostgreSQL, role-based access control, QR ticketing, event management, recruitment pipeline

I. INTRODUCTION

University clubs and co-curricular organisations are central to student life, providing platforms for leadership, creativity, and professional growth. At North South University (NSU), dozens of active clubs span academic, cultural, sports, and technology domains. Despite their importance, the management of club activities has historically depended on manual, decentralised processes: information spread through Facebook groups and physical notice boards; recruitment relied on unstructured online submissions or paper forms; and event RSVPs were tracked by hand in spreadsheets. These approaches caused students to miss events due to poor visibility, burdened administrators with repetitive administrative work, and provided the university with limited oversight of club operations.

NSU ClubHub was designed and developed to eliminate these inefficiencies. It is a comprehensive, full-stack web application that consolidates all club-related activities into one accessible digital ecosystem. The system is organised around three primary user roles—Students, Club Administrators, and Super Administrators—each of whom receives a tailored dashboard and a permission set that precisely matches their operational needs and responsibilities.

The key contributions of this project are:

- A centralised university club management portal that serves all stakeholder groups from a single, coherent interface.
- A fine-grained RBAC model with three distinct permission tiers enforced at both the API and UI layers.
- An event management module with QR code ticket generation and scan-based attendance verification, replacing paper check-in.
- An end-to-end automated recruitment pipeline covering cycle creation, application submission, administrative review, and real-time status tracking.
- An embedded AI assistant for in-platform guidance, frequently asked questions, and feature discovery.
- A persistent notification system that keeps all stakeholder groups informed of relevant activity in real time.

The remainder of this paper is organised as follows. Section II reviews related work in the area of university portals and club management systems. Section III describes the overall system architecture. Sections IV and V detail the frontend and backend technology stacks, respectively. Section VI presents the database design. Section VII covers the feature modules and REST API. Section VIII reports evaluation findings, and Section IX concludes the paper with directions for future work.

II. RELATED WORK

A. Commercial Club Management Platforms

Commercial platforms such as CampusGroups and OrgSync offer broad club management features including event calendars, membership tracking, and integrated communication tools. However, they are generic products designed for the widest possible market. As a result they are costly to license, difficult to customise for institution-specific workflows, and often lack features that reflect the practical realities of universities in developing regions [1]. Neither platform provides role-specific dashboards, automated recruitment pipelines, or QR-based ticket verification as first-class features.

B. Learning Management Systems

Moodle, Blackboard, and Microsoft SharePoint have been repurposed at several universities to host co-curricular content. These systems were, however, designed for academic course delivery and carry significant conceptual overhead when applied to club management [2]. Constructs such as recruitment cycles, application status tracking, club-wide announcements, and event ticketing require substantial workarounds or are simply unavailable.

C. Role-Based Portal Research

Research into university portal design consistently demonstrates that interfaces tailored to specific user roles improve both task efficiency and user satisfaction [3]. Users benefit from seeing only the tools and information relevant to their context, rather than navigating a feature-dense interface designed for all roles simultaneously. NSU ClubHub operationalises this finding through distinct dashboards and permission sets for each of its three roles.

D. RESTful University Systems

The adoption of RESTful architectures in university information systems has been shown to improve scalability, interoperability, and long-term maintainability compared to tightly coupled alternatives [4]. A clean separation between frontend and backend via well-defined API contracts also simplifies independent deployment and team-parallel development. NSU ClubHub follows this architectural pattern throughout.

E. Gap Analysis

Prior student projects at NSU and comparable institutions have produced partial solutions in isolation—event calendars, membership databases, or announcement boards—but none have integrated the full spectrum of club management into a unified, role-aware platform. NSU ClubHub addresses this gap by delivering a purpose-built, full-stack solution tailored to NSU’s operational context and scalable to peer institutions.

III. SYSTEM ARCHITECTURE

A. Three-Tier Overview

NSU ClubHub adopts a classic three-tier client-server architecture:

- 1) **Presentation layer** — Next.js frontend served to the user’s browser, responsible for all rendering and client-side interactivity.
- 2) **Application layer** — Express.js backend running on Node.js, handling routing, business logic, authentication, authorisation, and data orchestration.
- 3) **Data layer** — PostgreSQL relational database storing all persistent state, accessed exclusively through Prisma ORM.

Each tier is independently deployable, allowing horizontal scaling of the backend without affecting the frontend or database, and vice versa. The frontend communicates with the backend only through versioned RESTful API calls over HTTPS, ensuring a clean contract boundary between layers.

B. Repository Structure

The project is maintained as a monorepo with two top-level directories: `src/` for the Next.js frontend application, and `backend/` for the Express.js server. This structure enables coordinated version control and shared tooling while preserving the independence of each layer.

C. Security Architecture

Every protected API route is guarded by JWT middleware. On successful login, the server signs a JSON Web Token containing the user’s unique identifier and assigned role, using a secret key stored as an environment variable. The token carries an expiry claim to limit session lifetimes. All subsequent requests from the client must include the token in the `Authorization: Bearer` header. The middleware validates the signature, checks the expiry, and enforces the role-specific access policy before executing any handler.

Passwords are never stored in plaintext. The `bcryptjs` library hashes each password with a configurable salt factor before persistence. Even in the event of a database compromise, raw credentials remain unrecoverable.

D. Deployment Architecture

The Next.js frontend is deployable on Vercel, which provides built-in support for SSR, edge caching, and automatic HTTPS provisioning. The Express.js backend is designed for any Node.js-compatible host, such as Railway, Heroku, or a university-managed virtual private server. The PostgreSQL database can be hosted on managed cloud services such as Supabase or Neon, which provide automated backups, connection pooling, and monitoring dashboards. Environment-specific configuration, including database connection strings and JWT secrets, is injected at runtime via `dotenv` to keep sensitive values out of source control.

IV. FRONTEND TECHNOLOGY STACK

A. *Next.js 15*

The frontend is built on Next.js (v15.5.15+) with React (v19.0.0+). Next.js was chosen for its hybrid rendering capabilities: server-side rendering (SSR) delivers fully populated HTML to the browser on the initial request, accelerating perceived load times and improving SEO for publicly accessible pages such as the club directory and event listings. Static site generation (SSG) is applied where content changes infrequently, further reducing server load.

The project adopts the *App Router* structure introduced in Next.js 13 and stabilised in subsequent releases. Routes are defined by the file-system hierarchy under the `app/` directory. Nested layout files define persistent UI shells—navigation bars, sidebars, and authentication guards—that wrap entire route subtrees without re-rendering on navigation.

B. *Tailwind CSS*

Tailwind CSS provides a utility-first styling approach. Rather than authoring separate CSS files, developers compose styles inline using purpose-named utility classes. A centralised `tailwind.config` file defines the project’s design tokens—brand colours, spacing scale, and typography—ensuring visual consistency across all pages and components without the risk of style drift over time.

C. *Supporting Libraries*

Several third-party libraries complement the core framework:

- **Lucide React** — a consistent, tree-shakeable icon library providing over 1000 SVG icons used throughout the UI for visual affordance.
- **Radix UI** — a set of accessible, behaviour-complete UI primitives (dropdown menus, dialog modals, tooltips, popovers) that ship without visual styling, allowing full customisation with Tailwind while guaranteeing WCAG 2.1 keyboard and screen-reader compliance.
- **QRCode** — a client-side library that encodes a ticket identifier string into a scannable QR code image, rendered within the browser without any server round-trip.
- **Zod** — a TypeScript-first schema declaration and validation library used to define and enforce the shape of all user-submitted form data, providing immediate, field-level error feedback before any network request is made.

D. *State Management*

Global application state, particularly the authenticated user context and QR ticket data, is managed using React Context API combined with domain-specific custom hooks. Each hook encapsulates the data-fetching logic for its domain (clubs, events, notifications), exposes a clean interface to components, and handles loading and error states internally. This approach keeps components lean and avoids introducing the complexity of an external state management library such as Redux.

V. BACKEND TECHNOLOGY STACK

A. *Express.js with TypeScript*

The server is implemented using Express.js on Node.js, with TypeScript providing static typing across the entire codebase. TypeScript eliminates a broad class of runtime errors at compile time and substantially improves the developer experience through IDE autocomplete and refactoring support—particularly valuable in a team environment where multiple developers work across the same modules simultaneously.

The codebase is partitioned into domain-specific router files: `auth`, `clubs`, `events`, `recruitment`, `announcements`, `dashboard`, and `notifications`. Each router file registers its own set of routes and delegates to service-layer functions that contain the business logic. Shared middleware for authentication and role enforcement is imported centrally and applied to route groups using Express’s `router.use()` mechanism.

B. *Authentication and Validation*

Password hashing is performed with `bcryptjs` using a configurable salt factor; the default is set to 12 rounds, balancing security against computational cost. JWTs are signed with the `jsonwebtoken` library using the HS256 algorithm and a server-side secret. Tokens include an `exp` claim set to 24 hours by default.

Request payload validation is performed server-side with Zod schemas defined alongside each route handler. This creates a dual-layer validation strategy: the frontend rejects malformed submissions before they leave the browser, and the backend independently validates every incoming payload before processing it, guarding against direct API attacks that bypass the frontend entirely.

Cross-origin resource sharing (CORS) is configured with the `cors` middleware to accept requests only from the approved frontend origin, preventing cross-site request forgery from unauthorised domains.

C. *Prisma ORM*

Prisma is used as the sole data access layer between the Express.js application and PostgreSQL. The schema is declared in a single `schema.prisma` file using Prisma’s Schema Definition Language (SDL). From this file, Prisma generates a fully typed client library with auto-completed query methods for every model and relation.

Prisma Migrate generates and applies SQL migration files deterministically from schema changes. Each migration is stored in version control alongside the application code, providing a complete and reproducible history of all database changes across the project’s lifecycle.

VI. DATABASE DESIGN

A. *Choice of PostgreSQL*

PostgreSQL was selected as the primary RDBMS for its full ACID compliance, mature support for complex relational queries with multi-level joins, robust indexing options, and

TABLE I
CORE DATABASE MODELS AND KEY ATTRIBUTES

Model	Key Attributes
User	id, email, passwordHash, role, studentId, name
Club	id, name, category, description, contactEmail
Event	id, title, venue, dateTime, capacity, approvalStatus
RecruitmentCycle	id, clubId, title, startDate, endDate, isOpen
Application	id, cycleId, userId, status, submittedAt
Announcement	id, title, body, createdBy, createdAt
Notification	id, userId, message, isRead, createdAt
RSVP	userId, eventId, ticketQRCode, createdAt
ClubMember	userId, clubId, memberRole, joinedAt

widespread availability on managed cloud hosting platforms. Its strict schema enforcement also complements Prisma’s type-safe query model, reducing the risk of data inconsistency across the application.

B. Core Data Models

The database schema comprises nine principal models. Table I provides an overview of each model and its key attributes.

C. Relationships and Data Integrity

All entity relationships are enforced through foreign-key constraints managed by Prisma and the underlying PostgreSQL engine. Two explicit junction tables handle many-to-many relationships. `ClubMember` associates `User` records with `Club` records and stores the member’s role within that club (e.g., regular member, officer). `RSVP` associates students with events and carries the unique QR ticket code generated at the time of registration.

Events follow a two-phase publication model. A `Club Administrator` creates an event with an initial `approvalStatus` of `PENDING`. The event becomes visible to students only after a `Super Administrator` sets the status to `APPROVED`, preventing unreviewed or unauthorised content from reaching the platform. Cascading deletes are applied on parent-to-child relationships (e.g., deleting a club cascades to its events and recruitment cycles) to maintain referential integrity throughout the lifecycle of records.

VII. FEATURE MODULES AND API DESIGN

A. Role-Based Feature Set

Students can browse the complete club directory, view detailed club profiles, RSVP to approved events, download QR-code tickets to their dashboard, submit applications during open recruitment cycles, and monitor the status of each application. A personalised dashboard aggregates upcoming events, recent club announcements, and unread notifications in a single view.

Club Administrators access a dedicated management panel that enables them to: create and edit events (which

enter a pending approval queue); open, close, and manage recruitment cycles; review individual applications and update their status; post club-specific announcements; and maintain club membership records by adding or removing members and updating their roles.

Super Administrators exercise system-wide authority. They approve or reject events submitted by `Club Administrators`, publish platform-wide announcements visible to all users, manage all user accounts, assign and revoke roles, and access an administrative dashboard displaying system-wide engagement statistics.

B. QR-Based Ticketing

When a student RSVPs to an event, the backend generates a UUID that serves as the ticket identifier and stores it against the `RSVP` record in the database. The frontend receives this identifier and encodes it into a QR code image using the client-side `QRCode` library. The rendered QR code is displayed in the student’s dashboard under “My Tickets.” At event entry, club staff scan the QR code using any standard QR scanner; the system looks up the identifier, confirms its validity, and marks attendance, eliminating the need for printed lists or manual verification.

C. AI Assistant

An AI assistant is available via a persistent chat interface accessible from every dashboard. The assistant is designed to answer frequently asked questions about club membership, upcoming events, and the recruitment process. It can guide first-time users through platform workflows such as submitting an application or RSVPing to an event, reducing the burden on administrators who would otherwise field these queries individually.

D. Notification System

Notifications are generated server-side in response to key system events and persisted in the `Notification` table. Triggers include application status changes (submitted, under review, accepted, rejected), event approval decisions, new club announcements, and reminders for upcoming RSVPed events. The frontend polls the `/api/notifications` endpoint at a configurable interval and surfaces unread notifications in a badge indicator in the navigation bar.

E. REST API Reference

The backend exposes a comprehensive set of RESTful endpoints. Table II presents a representative subset grouped by domain. All endpoints that mutate state or return user-specific data require a valid JWT in the request header. Endpoints marked “SAdmin” are additionally restricted to the `SUPER_ADMIN` role.

F. User Interface Design

The platform employs a deliberate dual-theme strategy. The public landing page uses a dark, modern aesthetic with accent colours to attract prospective members who arrive from search engines or direct links. All authenticated dashboards switch

TABLE II
REPRESENTATIVE REST API ENDPOINTS

Endpoint	Method	Description
/api/auth/register	POST	Register a new user
/api/auth/login	POST	Authenticate; return JWT
/api/clubs	GET	List all clubs
/api/clubs/:id	GET	Get club details
/api/events	GET	List approved events
/api/events/:id/rsvp	POST	RSVP; issue QR ticket
/api/recruitment/:id/apply	POST	Submit an application
/api/admin/event/:id	PATCH	Approve event (SAdmin)
/api/notification	GET	Fetch user notifications
/api/dashboard/student	GET	Student dashboard data

to a clean light theme with high contrast, optimised for the data-dense, task-oriented interactions that administrators and students perform repeatedly.

Card-based layouts present clubs and events in a visually consistent, scannable format. Modal dialogs handle lightweight interactions—RSVP confirmations, quick application forms, and announcement previews—without navigating the user away from their current context. All layouts are fully responsive: a single codebase adapts fluidly to desktop, tablet, and mobile viewports using Tailwind’s responsive breakpoint utilities.

VIII. EVALUATION AND RESULTS

A. Functional Testing

The system was evaluated through structured testing covering all defined user stories across the three roles. Each functional module—authentication, club browsing, event RSVP, QR ticketing, recruitment application, and administrative controls—was tested against its acceptance criteria using a combination of manual walkthrough testing and automated API integration tests written with Jest and Supertest.

All critical user flows completed successfully. The full end-to-end event lifecycle was verified: a Club Administrator created an event, the Super Administrator approved it, a student RSVPed, and the QR code was generated and rendered in the student’s dashboard. The recruitment pipeline was similarly exercised from recruitment cycle creation, through student application submission and admin review, to final status update and student notification.

Edge cases tested included attempting to RSVP to a full event, submitting an application to a closed recruitment cycle, and accessing dashboards without an active session. All edge cases were handled correctly with appropriate HTTP status codes and user-facing error messages.

B. Security Testing

JWT validation was stress-tested by submitting expired tokens, cryptographically tampered tokens, tokens signed with an

incorrect secret, and requests containing no `Authorization` header. Every such request was correctly rejected with an HTTP 401 (Unauthorised) or HTTP 403 (Forbidden) response before any business logic was executed.

Role-based access control was validated by issuing student-level tokens against every admin-restricted endpoint and confirming that access was denied in all cases. Similarly, Club Administrator tokens were tested against Super Administrator endpoints. No privilege escalation was possible through any tested vector.

C. Performance Observations

Initial page load times on the authenticated student dashboard were measured at under 1.2 seconds on a standard 50 Mbps broadband connection, attributable to Next.js SSR pre-populating the page with critical data on the server before sending it to the browser. This eliminated the blank-screen period that would otherwise accompany a client-side data fetch.

REST API response times for list-style queries (clubs, events, notifications) averaged below 200 ms under a simulated concurrent load of 20 simultaneous users. Database query performance was monitored using Prisma’s query logging; no query exceeded 50 ms on the test dataset of 150 clubs, 400 events, and 1200 users.

D. Usability Feedback

Informal usability sessions were conducted with five NSU students recruited from different faculties. Participants were asked to complete three standardised tasks: (1) find a club, (2) RSVP to an upcoming event and locate the QR ticket, and (3) apply for a recruitment cycle. All five participants completed all three tasks without assistance within the allotted time.

Post-session feedback was consistently positive, particularly regarding the QR ticketing feature, which participants identified as a significant improvement over the current paper-based process. Suggestions for improvement included a richer onboarding experience for first-time users, enhanced search and filter capabilities in the club directory, and a calendar view for upcoming events.

E. Limitations

The current implementation uses HTTP polling for notifications, introducing a latency of up to 10 seconds between an event occurring and the notification appearing in the user’s browser. The AI assistant relies on a general-purpose language model and does not have access to live platform data, limiting the specificity of its answers. These limitations are acknowledged as targets for the next development iteration.

IX. CONCLUSION

This paper presented NSU ClubHub, a full-stack university club management portal that replaces fragmented, manual co-curricular processes with a secure, role-driven digital platform. By integrating club discovery, event management with QR-based ticketing, an automated recruitment pipeline, real-time notifications, and an AI assistant into a single cohesive system,

NSU ClubHub delivers measurable value to students, club administrators, and university management alike.

The platform is built on a modern and proven technology stack—Next.js and Tailwind CSS on the frontend; Express.js, TypeScript, and Prisma ORM on the backend; and PostgreSQL for persistent data storage—following industry best practices in RESTful API design, JWT-based authentication, and role-based access control. Functional testing verified correctness across all user flows, security testing confirmed robust access enforcement, and performance measurements demonstrated acceptable response times under realistic load conditions.

Several directions for future work are planned. First, the polling-based notification mechanism will be replaced with WebSocket connections to deliver truly real-time updates with sub-second latency. Second, native iOS and Android applications will be developed to improve accessibility for students who primarily interact with the university on mobile devices. Third, OAuth-based single sign-on (SSO) integration with NSU's institutional identity provider will simplify authentication and eliminate the need for separate credential management. Fourth, the AI assistant will be enhanced with retrieval-augmented generation (RAG) over a club-specific knowledge base, enabling it to answer questions about specific clubs, active recruitment cycles, and upcoming events with real data. Finally, engagement analytics dashboards will provide club administrators and university management with quantita-

tive insights into participation trends, event attendance rates, and recruitment funnel performance.

NSU ClubHub demonstrates that targeted, well-architected software engineering can effectively solve real organisational challenges in a university setting. The platform's modular design and clean technology choices ensure it can be extended, maintained, and adopted as a replicable model by peer institutions seeking to modernise their co-curricular management infrastructure.

REFERENCES

- [1] CampusGroups, "Campus Engagement Platform," *CampusGroups Inc.*, 2023. [Online]. Available: <https://www.campusgroups.com>
- [2] M. Dougiamas and P. Taylor, "Moodle: Using Learning Communities to Create an Open Source Course Management System," in *Proc. EdMedia World Conf. on Educational Media and Technology*, 2003, pp. 171–178.
- [3] R. Sandhu, E. Coyne, H. Feinstein, and C. Youman, "Role-Based Access Control Models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996.
- [4] R. T. Fielding and R. N. Taylor, "Principled Design of the Modern Web Architecture," *ACM Trans. Internet Technol.*, vol. 2, no. 2, pp. 115–150, May 2002.
- [5] Vercel, "Next.js Documentation," 2023. [Online]. Available: <https://nextjs.org/docs>
- [6] Prisma, "Prisma ORM Documentation," 2023. [Online]. Available: <https://www.prisma.io/docs>
- [7] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," *IETF RFC 7519*, May 2015.
- [8] Tailwind Labs, "Tailwind CSS Documentation," 2023. [Online]. Available: <https://tailwindcss.com/docs>